CoperniCoaches

Dia 1 - Lunes 16 de Marzo

Capgemini

Lunes 16 de marzo 2026

CoperniCoaches - Dia 1

Preparación Operativa

1. Formación de equipos
Se formarán equipos de 3–4 personas.
Intentaremos equilibrar cada equipo con perfiles distintos:

- experiencia previa con IA / Copilot
- perfil de analista
- perfil técnico o desarrollador

El objetivo es que todos los equipos tengan capacidades similares para trabajar durante el workshop.

2. Preparación del entorno de desarrollo
Cada participante deberá preparar su entorno siguiendo las guías disponibles en el portal C&CA (https://cca.capgemini.com)

Instalaciones necesarias:

- Git
- Node.js
- Python
- uv
- Visual Studio Code
- GitHub Copilot (acceso verificado)
- Editor de Graficos https://www.drawio.com/ o extension en VSCode https://github.com/hediet/vscode-drawio

Los instructores estarán disponibles para ayudar a resolver problemas de instalación.

3. Creación del repositorio del workshop

Durante la sesión crearemos un repositorio común en GitHub para el workshop.

Cada equipo deberá:

- clonar el repositorio en su máquina local
- crear su carpeta de equipo dentro del repositorio
- añadir un pequeño archivo de prueba (por ejemplo team.md)
- realizar su primer commit y push

Esto nos permitirá verificar que todos tienen acceso y que el entorno funciona correctamente.

4. Verificación final del entorno

Antes de cerrar el día cada participante debe comprobar que:

- Git funciona correctamente
- Node.js y Python están instalados
- puede trabajar con el repositorio del workshop
- Copilot está activo en su editor

Resultado esperado al final del día

Todos los equipos deben tener:

- entorno técnico operativo
- acceso al repositorio común
- carpeta de equipo creada en GitHub

Create cuenta Git

Preparación Operativa

Día 1 — Acceso al repositorio del workshop

Objetivo de este paso
Cada participante debe obtener acceso al repositorio del workshop y verificar que puede trabajar con él desde su máquina local.

1. Crear cuenta de GitHub (si aún no existe)
Si todavía no tienes cuenta:

1. Ir a github.com
2. Crear una cuenta personal
3. Verificar el email

Guía oficial en español:
https://docs.github.com/es/get-started/start-your-journey/creating-an-account-on-github

2. **Pasar el nombre de la nueva cuenta a Carlos o Iwan.** Se envian una invitacion a la dirección de correo asociada con la cuenta

3. **Aceptar la invitación al repositorio / organización**

Recibirás una invitación por correo o notificación de GitHub.

Pasos:

1. Abrir el correo o la notificación
2. Hacer clic en Accept Invitation
3. Confirmar que ya tienes acceso al repositorio del workshop

Guía:
docs.github.com/es/account-and-profile/how-tos/organization-membership/accepting-an-organization-invitation

4. Clonar el repositorio en tu máquina

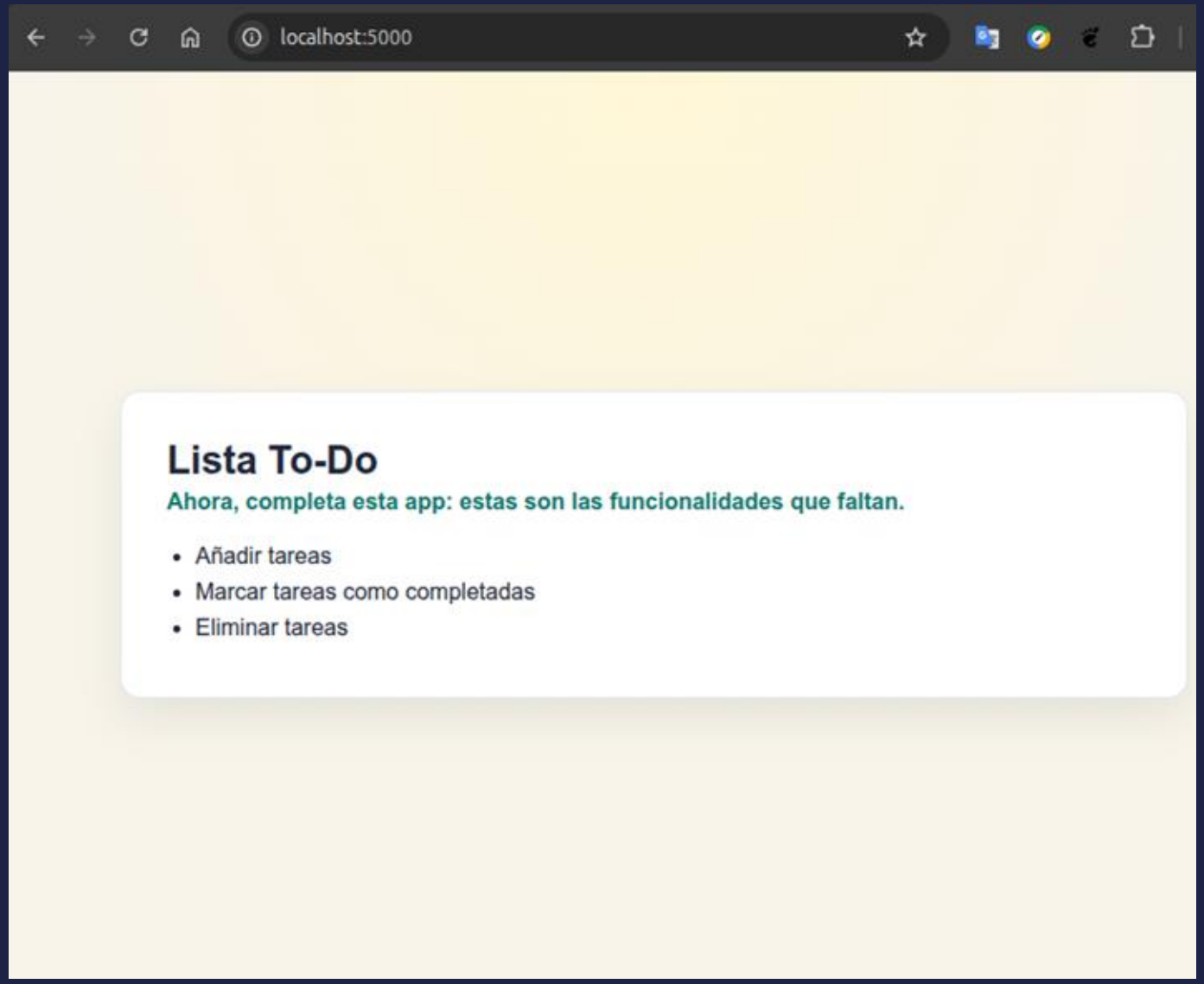
En la terminal:

git clone https://github.com/ThePhrontistery/copernicoaches

cd copernicoaches

Abrir el repositorio en tu editor (VS Code).

Ejercicio práctico



Ejercicio práctico 1/3

Objetivo

El objetivo de este ejercicio es comprobar que el entorno está listo y empezar a trabajar de forma ordenada con GitHub Copilot.

No se trata de programar rápido ni de improvisar.
Se trata de:

- clonar el repositorio
- poner en marcha el proyecto existente
- entender su estructura
- documentarlo
- definir una pequeña ampliación con una especificación clara
- implementar esa ampliación con ayuda de Copilot

Punto de partida

En el repositorio del workshop ya existe un proyecto base en:

dia1/

Ese proyecto contiene una aplicación Flask sencilla y un fichero pyproject.toml.

Vuestro trabajo consiste en partir de esa base.

Cada participante debe hacer un fork de este repositorio en su propia cuenta de GitHub en lugar de clonarlo directamente. Después de crear el fork, se debe clonar el repositorio desde vuestra propia cuenta y trabajar sobre esa copia.

Guía oficial en español que explica cómo hacer un fork:

https://docs.github.com/es/get-started/quickstart/fork-a-repo

2. Resolver la instalación del proyecto

El proyecto está preparado para trabajar con Python y uv.

Debéis conseguir que la aplicación funcione en local. Para ello tendréis que:

- revisar pyproject.toml
- instalar dependencias
- arrancar la aplicación
- verificar que abre correctamente en local

Podéis usar Copilot para ayudaros, pero debéis entender lo que estáis haciendo.

3. Analizar el proyecto existente

Antes de cambiar nada, debéis revisar la aplicación y entender:

qué hace ahora mismo , qué ficheros tiene, cómo está organizada, cómo se ejecuta y qué limitaciones tiene

Ejercicio práctico 2/3

4. Documentar el proyecto

Debéis crear un README.md breve dentro de dia1/ o de vuestra carpeta de equipo con al menos:

- propósito actual de la aplicación
- estructura básica del proyecto
- cómo instalar dependencias
- cómo ejecutarlo
- observaciones o problemas encontrados

5. Definir una ampliación: mini To-Do app

La aplicación actual debe ampliarse con una funcionalidad sencilla de tareas.

La ampliación debe permitir, como mínimo:

- ver tareas
- añadir tareas
- marcar tareas como hechas

Primero debéis escribir una pequeña especificación.

6. Escribir una spec antes de implementar

Antes de tocar el código, debéis crear un documento breve de especificación. Puede llamarse por ejemplo: todo-spec.md

La idea es evitar el “vibe coding”. Primero se piensa, luego se implementa.

La spec debe incluir, como mínimo:

- objetivo
- alcance
- comportamiento esperado
- cambios necesarios en la interfaz
- cambios necesarios en backend
- limitaciones
- criterios de aceptación

Pista para estructurar la spec

Podéis usar como referencia una estructura de technical specs. No hace falta hacer un documento largo, pero sí ordenado.

Ejemplo de apartados:

Objetivo
Alcance
Estado actual
Cambios funcionales
Cambios técnicos
Criterios de aceptación
Dudas abiertas

Ejercicio práctico 3/3

7. Crear un prompt útil para Copilot

Antes de implementar, debéis redactar un prompt claro para pedir ayuda a Copilot.

Ese prompt debe apoyarse en vuestra spec.
No debe ser algo genérico como “hazme un to-do app”.

Debe explicar:

- qué existe ya
- qué queréis añadir
- qué restricciones tenéis
- qué resultado esperáis

Qué se espera de Copilot

Podéis usar Copilot para:

- entender código existente
- proponer cambios
- resolver problemas de instalación
- generar partes de la implementación
- explicar errores

Pero no debéis limitaros a aceptar código sin criterio.
Tenéis que revisar lo que propone y comprobar que encaja con vuestra spec.

Entregables del día

Al final del ejercicio, cada equipo debe tener:

- proyecto funcionando en local
- README.md creado o actualizado
- spec breve para la ampliación To-Do
- prompt escrito para Copilot
- primera implementación de la funcionalidad To-Do, aunque sea parcial
- cambios subidos al repositorio

Resultado esperado

Al terminar el día, cada equipo habrá comprobado que:

- su entorno técnico funciona
- puede ejecutar un proyecto Python real
- puede analizar y documentar código existente
- puede usar Copilot de forma guiada
- puede trabajar con una pequeña spec antes de implementar

¡Gracias!

Company Confidential © Capgemini 2026. All rights reserved